

Competitive Programming Notebook

py.Sandu

Contents

1 DS	2
1.1 Fenwick Tree	2
1.2 Monotonic Stack	2
1.3 Union Find	2
1.4 Seg Tree	2
1.5 Trie	3
2 Math	4
2.1 Siege Of Eratosthenes	4
2.2 Gcd	4
2.3 Extended Euclidean Algorithm	4
2.4 Spf	4
2.5 Matrix Multiplication	4
2.6 Matrix Fibonacci	4
2.7 Solve Linear System	5
2.8 Modular Exponentiation	5
3 Graph	5
3.1 Bfs	5
3.2 Mst Prim	5
3.3 Reverse Graph	6
3.4 Floyd-warshall	6
3.5 Ford-fulkerson	6
3.6 Dfs	7
3.7 Eulerean Path	7
3.8 Bellman-ford	7
3.9 Dijkstra	7
4 DP	8
4.1 Longest Increasing Subsequence	8
4.2 Kadane Algorithm	8
4.3 Coins Change	8
4.4 Knapsack	8
5 String	8
5.1 Kmp	8
6 Geometry	9
6.1 Winding Number	9
6.2 Isleft	9
6.3 Cross Product	9
6.4 Convex Hull	9
6.5 Point	9
7 General	9

1 DS

1.1 Fenwick Tree

```

1 // Fenwick Tree to efficiently calculate and update
  prefix sums of an array of values.
2 // Time complexity: O(log N) for queries and updates
3
4 int sum(int i) {
5     int sum = 0;
6     while(i > 0) {
7         sum += T[i];
8         i -= i & -i; // flip the last bit
9     }
10    return sum;
11 }
12
13 void add(int i, int k) {
14     while (i < T.size()) {
15         T[i] += k;
16         i += i & -i; // add last set bit
17     }
18 }
19
20 vi make(vi arr) {
21     vi tree = arr;
22
23     for(int i = 0 ; i < tree.size() ; i++) {
24         int p = i + (i & -i) // index to parent
25         if (p < t.size()) {
26             tree[p] = tree[p] + tree[i];
27         }
28     }
29     return tree;
30 }

```

1.2 Monotonic Stack

```

1 // Increasing Monotonic Stack
2 // Time Complexity: O(n)
3 // Space Complexity: O(n)
4
5 vector<int> nextGreaterElements(vector<int>& arr) {
6     int n = arr.size();
7     vector<int> res(n, -1);
8     stack<int> st;
9
10    for (int i = 0; i < 2 * n; i++) {
11        int num = arr[i % n];
12        while (!st.empty() && arr[st.top()] < num) {
13            res[st.top()] = num;
14            st.pop();
15        }
16        if (i < n) {
17            st.push(i);
18        }
19    }
20
21    return res;
22 }
23
24 // Decreasing Monotonic Stack
25 // Time Complexity: O(n)
26 // Space Complexity: O(n)
27
28 vector<int> nextSmallerElements(vector<int>& arr) {
29     int n = arr.size();
30     vector<int> res(n, -1);
31     stack<int> st;
32
33     for (int i = 0; i < 2 * n; ++i) {
34         int num = arr[i % n];

```

```

35         while (!st.empty() && arr[st.top()] > num) {
36             res[st.top()] = num;
37             st.pop();
38         }
39         if (i < n) {
40             st.push(i);
41         }
42     }
43
44     return res;
45 }

```

1.3 Union Find

```

1 int find(vector<int>& union_find, int a) {
2     vector<int> path;
3
4     while(union_find[a] >= 0){
5         path.push_back(a);
6         a = union_find[a];
7     }
8
9     for(int vertice : path)
10        union_find[vertice] = a;
11
12    return a;
13 }
14
15 void unite(vector<int>& union_find, int a, int b) {
16     a = find(union_find, a);
17     b = find(union_find, b);
18
19     if(a == b) return;
20
21     if(abs(union_find[a]) < abs(union_find[b])) swap(
22         a, b);
23
24     union_find[a] += union_find[b];
25     union_find[b] = a;
26 }
27 // Example usage for Hamming distance problem
28
29 class Solution {
30 public:
31     int minimumHammingDistance(vector<int>& source,
32                                vector<int>& target, vector<vector<int>>&
33                                allowedSwaps) {
34         vector<int> union_find(source.size(), -1);
35         unordered_map<int, unordered_map<int, int>>
36         count;
37         for(int i = 0; i < allowedSwaps.size(); i++){
38             int a = allowedSwaps[i][0];
39             int b = allowedSwaps[i][1];
40             unite(union_find, a, b);
41         }
42         for(int i = 0; i < source.size(); i++){
43             int group = find(union_find, i);
44             count[group][source[i]]++;
45             count[group][target[i]]--;
46         }
47         int ans = 0;
48         for(auto& pair : count)
49             for(auto& freq : pair.second)
50                 ans += abs(freq.second);
51         return ans/2;
52 }
53 };

```

1.4 Seg Tree

```

1 // Segment Tree implementation

```

```

2 // Time complexity:  $O(\log n)$  for updates and queries
3
4 template<typename T>
5 class SegmentTree {
6 private:
7     int n;
8     vector<T> tree;
9     T combine(T a, T b) {
10         // Combine function for the segment tree (e.g
11         // sum, min, max)
12         return a + b; // Example: sum
13     }
14     void build(vector<T>& arr, int v, int tl, int tr)
15     {
16         if (tl == tr) {
17             tree[v] = arr[tl];
18         } else {
19             int tm = (tl + tr) / 2;
20             build(arr, v * 2, tl, tm);
21             build(arr, v * 2 + 1, tm + 1, tr);
22             tree[v] = combine(tree[v * 2], tree[v * 2
23             + 1]);
24         }
25     }
26     void update(int v, int tl, int tr, int pos, T
27     new_val) {
28         if (tl == tr) {
29             tree[v] = new_val;
30         } else {
31             int tm = (tl + tr) / 2;
32             if (pos <= tm) {
33                 update(v * 2, tl, tm, pos, new_val);
34             } else {
35                 update(v * 2 + 1, tm + 1, tr, pos,
36                 new_val);
37             }
38             tree[v] = combine(tree[v * 2], tree[v * 2
39             + 1]);
40         }
41     }
42     T query(int v, int tl, int tr, int l, int r) {
43         if (l > r) {
44             return T(); // Return identity element (e
45             .g., 0 for sum)
46         }
47         if (l == tl && r == tr) {
48             return tree[v];
49         }
50         int tm = (tl + tr) / 2;
51         return combine(query(v * 2, tl, tm, l, min(r,
52         tm)),
53         query(v * 2 + 1, tm + 1, tr,
54         max(l, tm + 1), r));
55     }
56 public:
57     SegmentTree(vector<T>& arr) {
58         n = arr.size();
59         tree.resize(4 * n);
60         build(arr, 1, 0, n - 1);
61     }
62     void update(int pos, T new_val) {
63         update(1, 0, n - 1, pos, new_val);
64     }
65     T query(int l, int r) {
66         return query(1, 0, n - 1, l, r);
67     }
68 };
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

1.5 Trie

```

1 // class for a node of the trie
2 class TrieNode {
3 public:

```

```

75         return;
76     }
77     node = node->children[index];
78 }
79 if (node->endofWord == true) {
80     node->endofWord = false;
81 }
82 }
83
84 // Function to print the trie
85 void print(TrieNode* node, string prefix) const
86 {
87     if (node->endofWord) {
88         cout << prefix << endl;
89     }
90     for (int i = 0; i < 26; i++) {
91         if (node->children[i]) {
92             print(node->children[i],
93                 prefix + char('a' + i));
94         }
95     }
96 }
97
98 // Function to start printing from the root
99 void print() const { print(root, ""); }
100 };

```

2 Math

2.1 Sieve Of Eratosthenes

```

1 // Find all prime numbers up to a given limit using
  the Sieve of Eratosthenes algorithm
2 // Time complexity:  $O(n \log \log n)$  where  $n$  is the
  limit
3
4 vector<int> findPrimes(int limit) {
5     vector<bool> isPrime(limit + 1, true);
6     isPrime[0] = isPrime[1] = false;
7
8     for (int i = 2; i * i <= limit; i++) {
9         if (isPrime[i]) {
10             for (int j = i * i; j <= limit; j += i) {
11                 isPrime[j] = false;
12             }
13         }
14     }
15
16     vector<int> primes;
17     for (int i = 2; i <= limit; i++) {
18         if (isPrime[i]) {
19             primes.push_back(i);
20         }
21     }
22     return primes;
23 }

```

2.2 Gcd

```

1 // Calculates the Greatest Common Divisor (GCD) of
  two integers using Euclidean algorithm
2 // Time Complexity:  $O(\log(\min(a, b)))$ 
3 // Space Complexity:  $O(\log(\min(a, b)))$ 
4
5 int gcd(int a, int b){
6     if (a == 0) return b;
7     return gcd(b % a, a);
8 }

```

2.3 Extended Euclidean Algorithm

```

1 // Extended Euclidean Algorithm
2 // Returns (gcd(a, b), x, y) such that  $a*x + b*y =$ 
  gcd(a, b)
3
4 int extendedEuclidean(int a, int b, int& x, int& y){
5     if(a == 0){
6         x = 0;
7         y = 1;
8         return b;
9     }
10    int x1, y1;
11    int gcd = extendedEuclidean(b % a, a, x1, y1);
12    x = y1 - (b / a) * x1;
13    y = x1;
14    return gcd;
15 }

```

2.4 Spf

```

1 // SPF sieve implementation for finding the smallest
  prime factor of each number up to a given limit.
2 // Time complexity:  $O(n \log \log n)$  for the sieve
  construction,  $O(\log n)$  for factorization of each
  number.
3
4 #define MAXN 1000000 // Maximum limit for the sieve
5 std::vector<int> spf(MAXN + 1);
6
7 void sieve() {
8     for (int i = 1; i <= MAXN; i++) {
9         spf[i] = i;
10    }
11    for (int i = 2; i * i <= MAXN; i++) {
12        if (spf[i] == i) {
13            for (int j = i * i; j <= MAXN; j += i) {
14                if (spf[j] == j) {
15                    spf[j] = i;
16                }
17            }
18        }
19    }
20 }

```

2.5 Matrix Multiplication

```

1 // Multiply two matrices A and B, and return the
  result in matrix C.
2 // Time Complexity:  $O(n^3)$ 
3 // Space Complexity:  $O(n^2)$ 
4
5 vector<vector<int>> matrixMultiplication(const vector
  <vector<int>>& A, const vector<vector<int>>& B) {
6     int n = A.size();
7     vector<vector<int>> C(n, vector<int>(n, 0)); //
  Initialize result matrix with zeros
8
9     for (int i = 0; i < n; i++) {
10        for (int j = 0; j < n; j++) {
11            for (int k = 0; k < n; k++) {
12                C[i][j] += A[i][k] * B[k][j];
13            }
14        }
15    }
16    return C;
17 }

```

2.6 Matrix Fibonacci

```

1 // Calculate the Nth Fibonacci number using matrix
  exponentiation.
2 // Time Complexity:  $O(\log N)$ 
3 // Space Complexity:  $O(1)$ 
4

```

```

5 void matrixExponentiation(vector<vector<long long>>&
  matrix, int power) {
6   int n = matrix.size();
7   vector<vector<long long>> result(n, vector<long
  long>(n, 0));
8
9   // Initialize result as the identity matrix
10  for (int i = 0; i < n; i++) {
11    result[i][i] = 1;
12  }
13
14  while (power) {
15    if (power % 2 == 1) {
16      matrixMultiplication(result, matrix,
  result);
17    }
18    matrixMultiplication(matrix, matrix, matrix);
19    power /= 2;
20  }
21
22  matrix = result;
23 }

```

2.7 Solve Linear System

```

1 // Solve a system of linear equations using LU
  decomposition
2 // Time complexity:  $O(n^3)$  for decomposition,  $O(n^2)$ 
  for forward and backward substitution
3
4 void luDecomposition(const vector<vector<double>>& A,
  vector<vector<double>>& L, vector<vector<double>
  >>& U) {
5   int n = A.size();
6   for (int i = 0; i < n; i++) {
7     for (int j = 0; j < n; j++) {
8       if (i <= j) {
9         U[i][j] = A[i][j];
10        for (int k = 0; k < i; k++) {
11          U[i][j] -= L[i][k] * U[k][j];
12        }
13      } else {
14        L[i][j] = A[i][j];
15        for (int k = 0; k < j; k++) {
16          L[i][j] -= L[i][k] * U[k][j];
17        }
18        L[i][j] /= U[j][j];
19      }
20    }
21  }
22 }
23
24 void forwardSubstitution(const vector<vector<double>
  >>& L, const vector<double>& b, vector<double>& y) {
25   int n = L.size();
26   for (int i = 0; i < n; i++) {
27     y[i] = b[i];
28     for (int j = 0; j < i; j++) {
29       y[i] -= L[i][j] * y[j];
30     }
31   }
32 }
33
34
35 void backwardSubstitution(const vector<vector<double>
  >>& U, const vector<double>& y, vector<double>& x) {
36   int n = U.size();
37   for (int i = n - 1; i >= 0; i--) {
38     x[i] = y[i];
39     for (int j = i + 1; j < n; j++) {
40       x[i] -= U[i][j] * x[j];

```

```

41     }
42     x[i] /= U[i][i];
43   }
44 }

```

2.8 Modular Exponentiation

```

1 // Calculates  $(x^n) \% MOD$ 
2 // Time Complexity:  $O(\log n)$ 
3 // Space Complexity:  $O(1)$ 
4
5 int powMod(int x, int n, int MOD){
6   int res = 1;
7
8   while(n >= 1){
9     if(n & 1){
10      res = (res * x) % MOD;
11      n--;
12    }
13    else{
14      x = (x * x) % MOD;
15      n /= 2;
16    }
17  }
18  return res;
19 }

```

3 Graph

3.1 Bfs

```

1 // Breadth First Search (BFS)
2 // Time Complexity:  $O(V + E)$ 
3 // Space Complexity:  $O(V)$ 
4 void bfs(const Graph& graph, int start) {
5   int V = graph.size();
6   vector<bool> visited(V, false);
7   queue<int> q;
8
9   visited[start] = true;
10  q.push(start);
11
12  while (!q.empty()) {
13    int vertex = q.front();
14    q.pop();
15    cout << vertex << " ";
16
17    for (int neighbor : graph[vertex]) {
18      if (!visited[neighbor]) {
19        visited[neighbor] = true;
20        q.push(neighbor);
21      }
22    }
23  }
24 }

```

3.2 Mst Prim

```

1 // Prim's algorithm for finding the Minimum Spanning
  Tree
2 // Time complexity:  $O(E \log V)$  where E is the number
  of edges and V is the number of vertices
3
4 struct Edge {
5   int u, v, weight;
6   bool operator<(const Edge& other) const {
7     return weight > other.weight; // Min-heap
8   }
9 };
10

```

```

11 vector<Edge> primMST(const vector<vector<int>>& graph
12 ) {
13     int n = graph.size();
14     vector<bool> visited(n, false);
15     vector<Edge> mst;
16     priority_queue<Edge> pq;
17
18     // Start with the first vertex
19     visited[0] = true;
20     for (int i = 0; i < n; ++i) {
21         if (graph[0][i] != 0) {
22             pq.push({0, i, graph[0][i]});
23         }
24     }
25
26     while (!pq.empty()) {
27         Edge current = pq.top();
28         pq.pop();
29
30         if (visited[current.v]) continue;
31
32         visited[current.v] = true;
33         mst.push_back(current);
34
35         for (int i = 0; i < n; ++i) {
36             if (!visited[i] && graph[current.v][i] !=
37                 0) {
38                 pq.push({current.v, i, graph[current.
39                     v][i]});
40             }
41         }
42     }
43
44     return mst;
45 }

```

3.3 Reverse Graph

```

1 // Reverse the edges of a directed graph
2 // Time Complexity: O(V + E)
3 // Space Complexity: O(V + E)
4
5 Graph reverseGraph(const Graph& graph) {
6     int V = graph.size();
7     Graph reversedGraph(V);
8
9     for (int u = 0; u < V; u++) {
10         for (int v : graph[u]) {
11             reversedGraph[v].push_back(u);
12         }
13     }
14
15     return reversedGraph;
16 }

```

3.4 Floyd-warshall

```

1 // Find the shortest paths in a weighted graph with
2 // positive or negative edge weights (but with no
3 // negative cycles).
4 // Time Complexity: O(V^3)
5 // Space Complexity: O(V^2)
6
7 void floydWarshall(Graph& graph, vector<vector<int>>&
8     dist) {
9     int V = graph.size();
10
11     for (int i = 0; i < V; i++) {
12         for (int j = 0; j < V; j++) {
13             if (i == j) {
14                 dist[i][j] = 0;
15             } else if (dist[i][j] == 0) {

```

```

16                 dist[i][j] = INT_MAX; // No edge
17             }
18         }
19     }
20
21     for (int k = 0; k < V; k++) {
22         for (int i = 0; i < V; i++) {
23             for (int j = 0; j < V; j++) {
24                 if (dist[i][k] != INT_MAX && dist[k][
25                     j] != INT_MAX) {
26                     dist[i][j] = min(dist[i][j], dist
27                         [i][k] + dist[k][j]);
28                 }
29             }
30         }
31     }
32 }

```

3.5 Ford-fulkerson

```

1 // Find maximum flow in graph
2 // Time complexity: O(|V| * E^2)
3 // Space complexity: O(V)
4 bool bfs(int residual[V][V], int start, int target,
5     vi parent){
6     vector<bool> visited(V, false);
7
8     queue<int> q;
9     q.push(start);
10    visited[start] = true;
11    parent[start] = -1;
12
13    while(!q.empty()){
14        int u = q.front();
15        q.pop();
16
17        for(int v = 0 ; v <= V ; v++){
18            if(!visited[v] and residual[u][v] > 0){
19                if(v == target){
20                    parent[v] = u;
21                    return true;
22                }
23                q.push(v);
24                parent[v] = u;
25                visited[v] = true;
26            }
27        }
28    }
29    return false;
30 }
31
32 int fordFulkerson(int g[V][V], int start, int target)
33 {
34     int u, v;
35     int residual[V][V];
36
37     for(u = 0 ; u < V ; u++){
38         for(v = 0 ; v < V ; v++){
39             residual[u][v] = g[u][v];
40         }
41     }
42
43     vi parent(V);
44     int maxFlow = 0;
45
46     while(bfs(residual, start, target, parent)){
47         int pathFlow = INT_MAX;
48         for(v = target ; v != start ; v = parent[v]){
49             u = parent[v];
50             pathFlow = min(pathFlow, residual[u][v]);
51         }
52
53         for(v = target ; v != start ; v = parent[v]){
54             u = parent[v];
55             residual[u][v] -= pathFlow;

```

```

52         residual[v][u] += pathFlow;
53     }
54     maxFlow += pathFlow;
55 }
56 return maxFlow;
57 }

```

3.6 Dfs

```

1 // Depth First Search (DFS)
2 // Time Complexity:  $O(V + E)$ 
3 // Space Complexity:  $O(V)$ 
4
5 // Recursive DFS
6 void dfs(const Graph& graph, int vertex, vector<bool>
    & visited) {
7     visited[vertex] = true;
8
9     for (int neighbor : graph[vertex]) {
10         if (!visited[neighbor]) {
11             dfs(graph, neighbor, visited);
12         }
13     }
14 }
15
16 // Iterative DFS
17 void dfsIterative(const Graph& graph, int start) {
18     int V = graph.size();
19     vector<bool> visited(V, false);
20     stack<int> s;
21     s.push(start);
22
23     while (!s.empty()) {
24         int vertex = s.top();
25         s.pop();
26
27         if (!visited[vertex]) {
28             visited[vertex] = true;
29
30             for (int neighbor : graph[vertex]) {
31                 if (!visited[neighbor]) {
32                     s.push(neighbor);
33                 }
34             }
35         }
36     }

```

3.7 Eulerean Path

```

1 // Find the Eulerian path in a graph using Hierholzer
  's algorithm
2 // Time complexity:  $O(E)$  where E is the number of
  edges in the graph
3
4 void findEulerianPath(const vector<vector<int>>&
    graph, int start, vector<int>& path) {
5     stack<int> s;
6     s.push(start);
7
8     while (!s.empty()) {
9         int v = s.top();
10        if (graph[v].empty()) {
11            path.push_back(v);
12            s.pop();
13        } else {
14            int u = graph[v].back();
15            graph[v].pop_back(); // Remove the edge
16            from v to u
17            s.push(u);
18        }
19    }

```

3.8 Bellman-ford

```

1 // Find the shortest path from a source vertex to all
  other vertices in a graph with negative weight
  edges and detect negative weight cycles.
2 // Time Complexity:  $O(V * E)$ 
3 // Space Complexity:  $O(V)$ 
4
5 void bellmanFord(Graph& graph, int src) {
6     int V = graph.size();
7     vector<int> dist(V, INT_MAX);
8     dist[src] = 0;
9
10    for (int i = 1; i < V; i++) {
11        for (int u = 0; u < V; u++) {
12            for (int v = 0; v < V; v++) {
13                if (graph[u][v] != 0 && dist[u] !=
14                    INT_MAX && dist[u] + graph[u][v] < dist[v]) {
15                    dist[v] = dist[u] + graph[u][v];
16                }
17            }
18        }
19    }
20
21    // Check for negative weight cycles
22    for (int u = 0; u < V; u++) {
23        for (int v = 0; v < V; v++) {
24            if (graph[u][v] != 0 && dist[u] !=
25                INT_MAX && dist[u] + graph[u][v] < dist[v]) {
26                cout << "Graph contains negative
27                    weight cycle" << endl;
28                return;
29            }
30        }
31    }
32 }

```

3.9 Djikstra

```

1 // Find shortest path from a source vertex to all
  other vertices in a graph with non-negative edge
  weights
2 // Time Complexity:  $O((V + E) \log V)$ 
3 // Space Complexity:  $O(V)$ 
4
5 void dijkstra(const Graph& graph, int src) {
6     int V = graph.size();
7     vector<int> dist(V, INT_MAX);
8     dist[src] = 0;
9
10    priority_queue<pair<int, int>, vector<pair<int,
11        int>>, greater<pair<int, int>>> pq; // Min-heap
12    priority queue
13    pq.push({0, src});
14
15    while (!pq.empty()) {
16        int u = pq.top().second;
17        pq.pop();
18
19        for (const auto& adj : graph[u]) {
20            int v = adj.first;
21            int weight = adj.second;
22
23            if (dist[u] != INT_MAX && dist[u] +
24                weight < dist[v]) {
25                dist[v] = dist[u] + weight;
26                pq.push({dist[v], v});
27            }
28        }
29    }
30 }

```

4 DP

4.1 Longest Increasing Subsequence

```

1 // Find the length of the longest increasing
  subsequence in a given array
2 // Time complexity:  $O(n \log n)$  where  $n$  is the length
  of the input array
3
4 int lis(vector<int>& arr) {
5     vector<int> dp;
6     for (int num : arr) {
7         auto it = lower_bound(dp.begin(), dp.end(),
8                               num);
9         if (it == dp.end()) {
10             dp.push_back(num);
11         } else {
12             *it = num;
13         }
14     }
15     return dp.size();
16 }

```

4.2 Kadane Algorithm

```

1 // Find maximum subarray sum using Kadane's algorithm
2 // Time complexity:  $O(n)$  where  $n$  is the length of the
  input array
3
4 int maxSubArraySum(const vector<int>& arr) {
5     int maxSum = arr[0];
6     int maxLocal = arr[0];
7
8     for (int i = 0; i < arr.size(); i++) {
9         maxLocal = max(arr[i], maxLocal + arr[i]);
10        maxSum = max(maxSum, maxLocal);
11    }
12
13    return maxSum;
14 }

```

4.3 Coins Change

```

1 // Find the minimum number of coins needed to make a
  given amount using a set of coin denominations
2 // Time complexity:  $O(n * m)$  where  $n$  is the amount
  and  $m$  is the number of coin denominations
3
4 int coinChange(const vector<int>& coins, int amount)
5 {
6     vector<int> dp(amount + 1, INT_MAX);
7     dp[0] = 0;
8
9     for (int i = 1; i <= amount; i++) {
10        for (const int& coin : coins) {
11            if (coin <= i && dp[i - coin] != INT_MAX)
12            {
13                dp[i] = min(dp[i], dp[i - coin] + 1);
14            }
15        }
16    }
17
18    return dp[amount] == INT_MAX ? -1 : dp[amount];
19 }

```

4.4 Knapsack

```

1 // Knapsack problem implementation using dynamic
  programming
2 // Time complexity:  $O(n*w)$ , where  $n$  is the number of
  items and  $w$  is the maximum weight

```

```

3 // Space complexity:  $O(n*w)$  for the dp table
4
5 int knapsack(int w, vector<ii>& items){
6     int n = items.size();
7     vector<vi> dp(n+1, vi(w+1, 0));
8
9     for(int i = 1 ; i <= n ; i++){
10        for(int j = 0 ; j <= w ; j++){
11            if(items[i-1].first > j) dp[i][j] = dp[i-1][j];
12            else dp[i][j] = max(dp[i-1][j], dp[i-1][j-1] + items[i-1].second);
13        }
14    }
15
16    return dp[n][w];
17 }

```

5 String

5.1 Kmp

```

1 // Find all occurrences of a pattern in a given text
  using the Knuth-Morris-Pratt (KMP) algorithm
2 // Time complexity:  $O(n + m)$  where  $n$  is the length of
  the text and  $m$  is the length of the pattern
3
4 vector<int> LPS(const string& pattern) {
5     int m = pattern.size();
6     vector<int> lps(m, 0);
7     int len = 0; // length of the previous longest
  prefix suffix
8     int i = 1;
9
10    while (i < m) {
11        if (pattern[i] == pattern[len]) {
12            len++;
13            lps[i] = len;
14            i++;
15        } else {
16            if (len != 0) {
17                len = lps[len - 1];
18            } else {
19                lps[i] = 0;
20                i++;
21            }
22        }
23    }
24
25    return lps;
26 }
27
28 vector<int> KMP(const string& text, const string&
  pattern) {
29     vector<int> lps = LPS(pattern);
30     vector<int> occurrences;
31     int n = text.size();
32     int m = pattern.size();
33     int i = 0; // index for text
34     int j = 0; // index for pattern
35
36    while (i < n) {
37        if (text[i] == pattern[j]) {
38            i++;
39            j++;
40        }
41        if (j == m) {
42            occurrences.push_back(i - j); // Pattern
  found at index (i - j)
43            j = lps[j - 1]; // Continue to search for
  the next match
44        } else if (i < n && text[i] != pattern[j]) {
45            if (j != 0) {

```

```

45         j = lps[j - 1];
46     } else {
47         i++;
48     }
49 }
50 }
51 return occurrences;
52 }

```

6 Geometry

6.1 Winding Number

```

1 // Check if a point is inside a polygon using the
  winding number algorithm
2
3 bool isInsidePolygon(const Point& p, const vector<
  Point>& polygon) {
4     int windingNumber = 0;
5     int n = polygon.size();
6
7     for (int i = 0; i < n; i++) {
8         const Point& v1 = polygon[i];
9         const Point& v2 = polygon[(i + 1) % n];
10
11         if (v1.y <= p.y) {
12             if (v2.y > p.y && isLeft(p, v1, v2)) {
13                 ++windingNumber;
14             }
15         } else {
16             if (v2.y <= p.y && !isLeft(p, v1, v2)) {
17                 --windingNumber;
18             }
19         }
20     }
21
22     return windingNumber != 0;
23 }

```

6.2 Isleft

```

1 // Check if a point is to the left of a segment
  defined by two points
2 bool isLeft(const Point& p, const Point& v1, const
  Point& v2) {
3     return (v2.x - v1.x) * (p.y - v1.y) - (v2.y - v1.
  y) * (p.x - v1.x) > 0;
4 }

```

6.3 Cross Product

```

1 double cross_product(Point O, Point A, Point B)
2 {
3     return (A.x - O.x) * (B.y - O.y)
4         - (A.y - O.y) * (B.x - O.x);
5 }

```

6.4 Convex Hull

```

1 // Returns a list of points on the convex hull
2 // in counter-clockwise order
3 vector<Point> convex_hull(vector<Point> A)
4 {
5     int n = A.size(), k = 0;
6
7     if (n <= 3)
8         return A;
9
10    vector<Point> ans(2 * n);
11
12    // Sort points lexicographically
13    sort(A.begin(), A.end());
14
15    // Build lower hull
16    for (int i = 0; i < n; ++i) {
17
18        // If the point at K-1 position is not a part
19        // of hull as vector from ans[k-2] to ans[k
20        -1]
21        // and ans[k-2] to A[i] has a clockwise turn
22        while (
23            k >= 2
24            && cross_product(ans[k - 2], ans[k - 1],
25                A[i])
26                <= 0)
27            k--;
28            ans[k++] = A[i];
29        }
30
31    // Build upper hull
32    for (size_t i = n - 1, t = k + 1; i > 0; --i) {
33
34        // If the point at K-1 position is not a part
35        // of hull as vector from ans[k-2] to ans[k
36        -1]
37        // and ans[k-2] to A[i] has a clockwise turn
38        while (k >= t
39            && cross_product(ans[k - 2], ans[k -
40                1],
41                    A[i - 1])
42                    <= 0)
43            k--;
44            ans[k++] = A[i - 1];
45        }
46
47    // Resize the array to desired size
48    ans.resize(k - 1);
49
50    return ans;
51 }

```

6.5 Point

```

1 struct Point {
2     double x, y;
3 };

```

7 General